

# VNBOT BACKEND

Servicios, módulos, API, eventos, jobs, scheduler, memoria, grafo, agentes y MCP.

Especificación técnica del backend de VNBOT: estructura de servicios, contextos de dominio, API pública versionada, motor de recordatorios, jobs y colas, audio, agentes, MCP Gateway, persistencia, healthchecks y reglas de seguridad. El cliente solicita, la API autentica, el dominio decide, la cola ejecuta, las integraciones actúan con permisos y la auditoría registra el resultado.

STATELESS API

IDEMPOTENT JOBS

POLICY ENGINE

AUDITABLE

# Tabla de Contenidos

---

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>1. Propósito</b>                             | <b>7</b>  |
| <b>2. Objetivos del backend</b>                 | <b>7</b>  |
| 2.1. Objetivos funcionales . . . . .            | 7         |
| 2.2. Objetivos operativos . . . . .             | 7         |
| 2.3. No objetivos del backend inicial . . . . . | 8         |
| <b>3. Arquitectura de servicios</b>             | <b>8</b>  |
| 3.1. Vista general . . . . .                    | 8         |
| 3.2. Servicios del MVP . . . . .                | 8         |
| 3.3. Servicios posteriores . . . . .            | 9         |
| <b>4. Estructura de módulos del API</b>         | <b>9</b>  |
| 4.1. Regla de dependencias . . . . .            | 9         |
| <b>5. Contextos de dominio</b>                  | <b>9</b>  |
| <b>6. Identidad, usuarios y workspaces</b>      | <b>10</b> |
| 6.1. Usuario . . . . .                          | 10        |
| 6.2. Workspace . . . . .                        | 10        |
| 6.3. Roles . . . . .                            | 10        |
| 6.4. Autorización . . . . .                     | 10        |
| <b>7. Modelo de operaciones</b>                 | <b>10</b> |
| 7.1. Propósito . . . . .                        | 10        |
| 7.2. Estados . . . . .                          | 11        |

|                                                     |           |
|-----------------------------------------------------|-----------|
| 7.3. Idempotencia . . . . .                         | 11        |
| 7.4. Expiración . . . . .                           | 11        |
| <b>8. API pública versionada</b>                    | <b>11</b> |
| <b>9. Endpoints de autenticación</b>                | <b>11</b> |
| <b>10. Endpoints de chat</b>                        | <b>12</b> |
| <b>11. Endpoints de memoria</b>                     | <b>13</b> |
| <b>12. Endpoints del grafo</b>                      | <b>14</b> |
| <b>13. Endpoints de recordatorios</b>               | <b>14</b> |
| <b>14. Motor de recordatorios</b>                   | <b>15</b> |
| 14.1. Separación entre regla y ocurrencia . . . . . | 15        |
| 14.2. Scheduler . . . . .                           | 15        |
| 14.3. Delivery worker . . . . .                     | 15        |
| 14.4. Canales . . . . .                             | 16        |
| <b>15. Jobs y cola</b>                              | <b>16</b> |
| 15.1. Tipos de job . . . . .                        | 16        |
| 15.2. Estructura . . . . .                          | 16        |
| 15.3. Prioridades . . . . .                         | 16        |
| 15.4. Dead-letter queue . . . . .                   | 16        |
| <b>16. Audio y archivos</b>                         | <b>17</b> |
| 16.1. Endpoint de subida . . . . .                  | 17        |
| 16.2. Audio — estados . . . . .                     | 17        |
| 16.3. Política de retención . . . . .               | 17        |

|                                            |           |
|--------------------------------------------|-----------|
| 16.4. Documentos e imágenes . . . . .      | 17        |
| <b>17. Agentes, skills y policy engine</b> | <b>17</b> |
| 17.1. Agente . . . . .                     | 17        |
| 17.2. Skill . . . . .                      | 18        |
| 17.3. Policy decision . . . . .            | 18        |
| 17.4. Tool call . . . . .                  | 18        |
| <b>18. MCP Gateway</b>                     | <b>18</b> |
| 18.1. Registro . . . . .                   | 18        |
| 18.2. Handshake . . . . .                  | 18        |
| 18.3. Scopes iniciales . . . . .           | 19        |
| 18.4. Aislamiento . . . . .                | 19        |
| <b>19. LLM Router y contexto</b>           | <b>19</b> |
| 19.1. Pipeline . . . . .                   | 19        |
| 19.2. Contexto mínimo . . . . .            | 19        |
| 19.3. Structured output . . . . .          | 20        |
| <b>20. Persistencia y repositorios</b>     | <b>20</b> |
| 20.1. Repositories . . . . .               | 20        |
| 20.2. Transacciones . . . . .              | 20        |
| 20.3. Soft delete . . . . .                | 20        |
| 20.4. Migraciones . . . . .                | 20        |
| <b>21. Caching e índices</b>               | <b>21</b> |
| 21.1. Redis . . . . .                      | 21        |
| 21.2. Índices de base de datos . . . . .   | 21        |

|                                               |           |
|-----------------------------------------------|-----------|
| 21.3. Índice vectorial . . . . .              | 21        |
| <b>22. Notificaciones</b>                     | <b>21</b> |
| 22.1. Modelo . . . . .                        | 21        |
| 22.2. Estados . . . . .                       | 21        |
| 22.3. Reglas . . . . .                        | 21        |
| <b>23. Exportación, importación y backups</b> | <b>22</b> |
| 23.1. Exportación . . . . .                   | 22        |
| 23.2. Importación . . . . .                   | 22        |
| 23.3. Backup . . . . .                        | 22        |
| <b>24. Healthchecks</b>                       | <b>22</b> |
| <b>25. Errores y códigos</b>                  | <b>23</b> |
| 25.1. Categorías . . . . .                    | 23        |
| 25.2. Ejemplos . . . . .                      | 23        |
| 25.3. Retryable . . . . .                     | 23        |
| <b>26. Seguridad del backend</b>              | <b>23</b> |
| 26.1. Secretos . . . . .                      | 23        |
| 26.2. Archivos . . . . .                      | 23        |
| 26.3. URLs externas . . . . .                 | 24        |
| 26.4. Prompt injection . . . . .              | 24        |
| <b>27. Concurrencia y consistencia</b>        | <b>24</b> |
| 27.1. Recordatorios . . . . .                 | 24        |
| 27.2. Memoria . . . . .                       | 24        |

|                                                |           |
|------------------------------------------------|-----------|
| 27.3. Integraciones . . . . .                  | 24        |
| <b>28. Escalabilidad</b>                       | <b>25</b> |
| 28.1. Escalado vertical inicial . . . . .      | 25        |
| 28.2. Escalado horizontal . . . . .            | 25        |
| 28.3. Cuellos de botella esperados . . . . .   | 25        |
| 28.4. Límites configurables . . . . .          | 25        |
| <b>29. Configuración</b>                       | <b>26</b> |
| 29.1. Variables principales . . . . .          | 26        |
| 29.2. Reglas . . . . .                         | 26        |
| <b>30. Pruebas del backend</b>                 | <b>26</b> |
| 30.1. Unitarias . . . . .                      | 26        |
| 30.2. Integración . . . . .                    | 26        |
| 30.3. Contratos . . . . .                      | 27        |
| 30.4. E2E . . . . .                            | 27        |
| <b>31. Observabilidad y privacidad</b>         | <b>27</b> |
| 31.1. Se puede registrar . . . . .             | 27        |
| 31.2. No registrar por defecto . . . . .       | 27        |
| <b>32. Requisitos para Docker</b>              | <b>28</b> |
| 32.1. Volúmenes . . . . .                      | 28        |
| <b>33. Criterios de aceptación backend MVP</b> | <b>28</b> |
| <b>34. Orden de implementación backend</b>     | <b>29</b> |
| <b>35. Decisiones abiertas</b>                 | <b>29</b> |

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>36. Conclusión</b>                              | <b>30</b> |
| <b>37. Endpoints de sincronización (fase 0.3+)</b> | <b>30</b> |
| <b>38. Requisitos de testing del backend</b>       | <b>31</b> |
| 38.1. Tests unitarios . . . . .                    | 31        |
| 38.2. Tests de integración . . . . .               | 31        |
| 38.3. Contract testing para MCP . . . . .          | 31        |
| 38.4. Benchmarks automatizados . . . . .           | 31        |
| <b>39. Instrumentación de observabilidad</b>       | <b>32</b> |
| 39.1. Spans obligatorios . . . . .                 | 32        |
| 39.2. Métricas obligatorias . . . . .              | 32        |
| 39.3. Regla de contenido . . . . .                 | 32        |
| <b>40. Restricciones de rendimiento del grafo</b>  | <b>32</b> |
| 40.1. Límites por consulta . . . . .               | 32        |
| 40.2. No cargar el grafo completo . . . . .        | 32        |
| 40.3. Degradación graciosa . . . . .               | 33        |

# 1. Propósito

Este documento especifica la estructura del backend de VNBOT: servicios, módulos, entidades, API, eventos, trabajos asíncronos, scheduler, memoria, grafo, agentes, MCP, notificaciones, archivos, healthchecks, errores y reglas de seguridad. Es la referencia detallada que falta entre el TRD (decisión técnica) y la implementación concreta de código.

El backend debe soportar cuatro formas de ejecución, cada una con componentes distintos:

- **Local:** un usuario, SQLite y servicios mínimos.
- **Servidor personal:** varios dispositivos, PostgreSQL, Redis y workers.
- **Servidor multiusuario:** workspaces aislados y servicios replicables.
- **Modo híbrido:** datos locales con sincronización opcional.

El cliente solicita. La API autentica y valida. El dominio decide. La cola ejecuta trabajos durables. Las integraciones actúan con permisos. La auditoría registra el resultado.

## 2. Objetivos del backend

### 2.1. Objetivos funcionales

- Gestionar usuarios, sesiones y workspaces.
- Procesar mensajes de chat.
- Interpretar intenciones mediante LLM o heurística.
- Guardar memorias y relaciones.
- Consultar el grafo personal.
- Crear recordatorios persistentes.
- Ejecutar notificaciones sin duplicados.
- Procesar audio y archivos de forma asíncrona.
- Gestionar agentes y skills.
- Conectar herramientas MCP.
- Mantener auditoría de acciones.
- Exportar e importar datos.
- Exponer healthchecks.

### 2.2. Objetivos operativos

- Poder reiniciar API y workers sin perder trabajos confirmados.
- Permitir reintentos seguros.



- Poder sustituir SQLite por PostgreSQL.
- Poder reemplazar Redis por una implementación local limitada.
- Poder añadir proveedores LLM sin cambiar la lógica de dominio.
- Poder añadir integraciones como plugins/adaptadores.
- Poder ejecutar la aplicación en Docker.

### 2.3. No objetivos del backend inicial

- Ejecutar código arbitrario del usuario.
- Automatización financiera.
- Scraping de cuentas personales.
- Escucha permanente del micrófono.
- Acceso universal al filesystem.
- Envío automático de mensajes a terceros sin consentimiento.
- Orquestación multiagente totalmente autónoma.

## 3. Arquitectura de servicios

### 3.1. Vista general

La arquitectura del backend separa la API stateless de los workers durables y del scheduler idempotente. Esta separación permite reiniciar cualquier componente sin perder trabajos confirmados y permite escalar horizontalmente solo los componentes que lo necesiten.

### 3.2. Servicios del MVP

| Servicio          | Responsabilidad                                                                                                                |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------|
| vnbot-api         | Servicio HTTP principal. Gestiona autenticación, comandos, consultas, confirmaciones, memoria, recordatorios y administración. |
| vnbot-worker      | Procesa tareas largas o reintentables: audio, embeddings, notificaciones, consolidación, backups y sincronización.             |
| vnbot-scheduler   | Calcula qué ocurrencias de recordatorios deben encolarse. Debe ejecutarse como instancia única o usar locks distribuidos.      |
| vnbot-mcp-gateway | Gestiona servidores MCP externos, schemas, scopes, timeouts, policy engine y auditoría.                                        |

En una instalación local estos servicios pueden ejecutarse dentro de un proceso supervisado, pero deben mantener límites de módulos claros para permitir la separación cuando se mueva a servidor.

### 3.3. Servicios posteriores

- notification-service dedicado.
- transcription-service separado si se necesita GPU.
- embedding-service local.
- sync-service multi-dispositivo.
- community-plugin-registry.
- observability-stack.

No se deben extraer microservicios únicamente por moda. La separación se hará cuando exista una necesidad de escalado, seguridad o aislamiento real.

## 4. Estructura de módulos del API

El servicio API se organiza en una arquitectura hexagonal con separación clara entre capas. La regla de dependencias es estricta: el dominio nunca debe importar frameworks de infraestructura.

```
services/api/<br/>└─ app<br/>└─ main.py<br/>└─ config.py<br/>└─ dependencies.py<br/>
```

### 4.1. Regla de dependencias

```
api → application → domain<br/>infrastructure → application/domain ports
```

El dominio no debe importar FastAPI, Redis, SQLAlchemy, proveedores LLM ni SDKs externos. Esta regla protege el núcleo de cambios de infraestructura.

## 5. Contextos de dominio

El dominio se divide en 7 contextos delimitados siguiendo Domain-Driven Design. Cada contexto tiene sus propias entidades, reglas y repositorios. Los contextos se comunican mediante eventos y comandos tipados, no mediante llamadas directas acopladas.

| Contexto | Gestiona                                                                                                     |
|----------|--------------------------------------------------------------------------------------------------------------|
| Identity | Usuarios, sesiones, workspaces, roles, MFA posterior, recuperación de cuenta.                                |
| Memory   | Memorias, nodos, aristas, procedencia, confianza, sensibilidad, expiración, correcciones, borrado.           |
| Reminder | Recordatorios, reglas de recurrencia, ocurrencias, canales, estados de entrega, posponer/completar/cancelar. |
| Agent    | Agentes, prompts, modelos, skills, presupuestos, nivel de autonomía, herramientas y permisos.                |

| Contexto    | Gestiona                                                                  |
|-------------|---------------------------------------------------------------------------|
| Operation   | Ciclo de vida de toda acción iniciada por usuario o agente.               |
| Integration | Integraciones, OAuth, tokens, scopes, healthchecks, revocación.           |
| File        | Audios, imágenes, documentos, retención, cifrado, referencias a memorias. |

## 6. Identidad, usuarios y workspaces

### 6.1. Usuario

```
User<br/>- id<br/>- email nullable en modo local<br/>- display_name<br/>- password_hash nullable<br/>-
```

### 6.2. Workspace

Un workspace separa contextos y permisos. Permite que un usuario tenga espacios distintos para Personal, Trabajo, Estudio, Familia o Proyecto VNBOT sin mezclar datos.

```
Workspace<br/>- id<br/>- owner_user_id<br/>- name<br/>- type<br/>- settings_json<br/>- created_at<br/>-
```

### 6.3. Roles

- **owner** — propietario del workspace, control total.
- **admin** — administración sin poder eliminar el workspace.
- **member** — uso completo de memoria y recordatorios.
- **viewer** — solo lectura.

El modo personal puede utilizar solo owner, pero el modelo debe permitir expansión para workspaces compartidos futuros.

### 6.4. Autorización

Cada endpoint debe obtener el workspace desde la sesión y verificar que el recurso pertenece a dicho workspace. Nunca se debe aceptar solamente un workspace\_id enviado por el cliente.

## 7. Modelo de operaciones

### 7.1. Propósito

Una operación representa una intención completa: crear un recordatorio, guardar una memoria, ejecutar una herramienta o procesar audio. El modelo de operaciones da trazabilidad completa de cada acción del usuario y del agente.



- Aplicar rate limit.
- Hash Argon2id.
- No revelar si un email existe en contextos sensibles.
- Enviar verificación si el despliegue lo requiere.

## POST /auth/login

Crea sesión segura y devuelve cookies o tokens según el cliente.

## POST /auth/logout

Revoca la sesión actual.

## POST /auth/refresh

Rota refresh token cuando aplique.

GET /auth/me

Devuelve usuario y workspaces permitidos, nunca secretos.

## POST /auth/lock

Bloquea la bóveda de la sesión local o solicita reautenticación.

## 10. Endpoints de chat

POST /chat

Entrada:

Proceso:

01. Validar usuario y workspace.
02. Crear mensaje.
03. Crear operación.
04. Clasificar intención.
05. Recuperar contexto autorizado.
06. Generar propuesta estructurada.
07. Evaluar riesgo.
08. Responder con texto y/o operation\_id.

Respuesta posible:

## GET /chat/conversations

Lista conversaciones paginadas del workspace.

```
GET /chat/conversations/{id}
```

Devuelve mensajes con paginación. Las partes sensibles deben respetar la política de retención.

```
POST /chat/operations/{id}/confirm
```

Confirma una propuesta. La API debe volver a validar permisos, fechas y disponibilidad antes de ejecutar.

```
POST /chat/operations/{id}/cancel
```

Cancela una propuesta o job si todavía es cancelable.

## 11. Endpoints de memoria

```
POST /memory/nodes
```

Crea un nodo explícito. Ejemplo:

```
{<br/>&nbsp;&nbsp;&nbsp;"type": "preference",<br/>&nbsp;&nbsp;&nbsp;"label": "Horario preferido de reuniones",<br/>&nbsp;&nbsp;&nbsp;"sensitivity": 100,<br/>&nbsp;&nbsp;&nbsp;"created_after": "2023-01-01T00:00:00Z"}
```

```
GET /memory/nodes/{id}
```

Devuelve nodo, procedencia y relaciones autorizadas.

```
PATCH /memory/nodes/{id}
```

Permite corregir contenido, label, tipo, sensibilidad o expiración. Toda modificación crea un evento de auditoría.

```
DELETE /memory/nodes/{id}
```

Debe aplicar política de borrado:

- Marcar como eliminado.
- Eliminar relaciones dependientes.
- Invalidar índices.
- Programar borrado de embeddings y archivos relacionados.
- Confirmar al usuario qué fue eliminado.

```
GET /memory/search
```

Parámetros:

```
q<br/>mode=text|semantic|hybrid|graph<br/>limit<br/>cursor<br/>type<br/>sensitivity<br/>created_after<br/>created_before
```

La respuesta debe incluir relevancia, fuente y nivel de confianza.

```
POST /memory/edges
```

Crea relación entre nodos. Las aristas inferidas deben marcarse como inferred y no como explicit.

```
DELETE /memory/edges/{id}
```

Elimina o invalida una relación.

## 12. Endpoints del grafo

GET /graph/subgraph

Entrada:

Respuesta:

GET /graph/stats

Devuelve contadores agregados del workspace:

- Total de nodos.
- Total de relaciones.
- Tipos.
- Memorias expiradas.
- Índices pendientes.

POST /graph/rebuild

Solo para administración o usuario autorizado. Encola reindexación; no bloquea la API.

POST /graph/merge

Fusiona entidades duplicadas con confirmación. Debe conservar procedencia e historial.

### 13. Endpoints de recordatorios

## POST /reminders

Crea un recordatorio con una regla explícita:

## GET /reminders

Filtros disponibles:

GET /reminders/{id}

Incluye regla, próxima ocurrencia, historial y estado.

```
PATCH /reminders/{id}
```

Toda modificación debe actualizar futuras ocurrencias sin alterar entregas históricas.

POST /reminders/{id}/complete

Completa la ocurrencia actual o el recordatorio completo según el payload.

POST /reminders/{id}/snooze

Entrada:

POST /reminders/{id}/cancel

Cancela futuras ocurrencias y conserva historial.

GET /reminders/{id}/deliveries

Muestra intentos y resultados sin incluir credenciales ni contenido innecesario.

## 14. Motor de recordatorios

### 14.1. Separación entre regla y ocurrencia

Un recordatorio recurrente no debe duplicarse como cientos de filas sin control. La separación entre regla (Recurrence Rule) y ocurrencias (Occurrences) permite generar instancias con anticipación controlada.

Las ocurrencias pueden crearse con una ventana anticipada, por ejemplo 30 días, y generarse progresivamente por el scheduler.

## 14.2. Scheduler

Cada ciclo del scheduler ejecuta 5 pasos:

01. Busca recordatorios activos.
02. Calcula ocurrencias próximas según timezone.
03. Adquiere lock.
04. Crea job idempotente.
05. Libera lock.

### 14.3. Delivery worker

El worker de entrega sigue 8 pasos:

01. Toma job.
02. Comprueba que la ocurrencia sigue activa.
03. Comprueba que no fue entregada.



04. Comprueba ventana de silencio.

05. Selecciona canal.

06. Envía notificación.

07. Registra resultado.

08. Reintenta si el error es temporal.

## 14.4. Canales

Cada canal implementa una interfaz común:

```
class NotificationChannel(Protocol):  
    ...  
    async def send(self, notification: Notification):
```

# 15. Jobs y cola

## 15.1. Tipos de job

El sistema maneja 14 tipos de jobs asíncronos:

```
transcribe_audio  
extract_document  
embed_memory  
index_memory  
consolidate_memory  
send_reminder
```

## 15.2. Estructura

Estructura JSON de un job en cola:

```
{  
    "id": "job_123",  
    "type": "send_reminder",  
    "payload": {  
        ...  
    }  
}
```

## 15.3. Prioridades

| Prioridad | Uso                                     |
|-----------|-----------------------------------------|
| critical  | Operaciones de seguridad o restauración |
| high      | Recordatorios próximos                  |
| normal    | Transcripción y embeddings              |
| low       | Consolidación y estadísticas            |

## 15.4. Dead-letter queue

Un job que supera sus reintentos pasa a DLQ. El usuario o administrador puede ver el error resumido, reintentar, cancelar, corregir configuración o exportar el incidente para análisis.

## 16. Audio y archivos

### 16.1. Endpoint de subida

```
POST /files/upload
```

Recibe un archivo temporal con:

- Tamaño máximo.
- MIME permitido.
- Hash.
- Usuario/workspace.
- Política de retención.

Nunca se debe confiar solo en la extensión del archivo. El MIME type debe validarse realmente con magic bytes, no solo con el Content-Type header.

### 16.2. Audio — estados

Pipeline de estados del audio:

```
uploaded<br/> validated<br/> queued_transcription<br/> transcribing<br/> transcript_ready<br/> us
```

### 16.3. Política de retención

El usuario debe elegir entre 4 opciones:

- Eliminar audio después de transcribir.
- Conservar audio junto a la memoria.
- Conservar durante un periodo.
- No guardar audio en ningún servidor.

### 16.4. Documentos e imágenes

Los documentos deben procesarse en worker. El OCR o extracción no debe bloquear la request HTTP. La respuesta inicial debe devolver job\_id y estado para que el cliente pueda polling o recibir WebSocket update.

## 17. Agentes, skills y policy engine

### 17.1. Agente

```
Agent<br/>- id<br/>- workspace_id<br/>- name<br/>- description<br/>- system_prompt<br/>- model_config
```

## 17.2. Skill

```
Skill<br/>- id<br/>- version<br/>- name<br/>- input_schema<br/>- output_schema<br/>- risk_level<br/>-
```

## 17.3. Policy decision

El policy engine devuelve decisiones estructuradas:

```
{<br/>&nbsp;&nbsp;&nbsp;"allowed": false,<br/>&nbsp;&nbsp;&nbsp;"reason": "TOOL_REQUIRES_CONFIRMATION",<br/>&nbsp;&nbsp;&nbsp;
```

## 17.4. Tool call

Cada llamada a herramienta debe registrar:

- Agente.
- Skill.
- Herramienta.
- Input filtrado.
- Scope.
- Resultado.
- Duración.
- Error.
- Confirmación.

Los inputs completos pueden omitirse o cifrarse si contienen datos sensibles. La auditoría no debe convertirse en una filtración de datos privados.

# 18. MCP Gateway

## 18.1. Registro

```
POST /integrations/mcp
```

Entrada:

```
{<br/>&nbsp;&nbsp;&nbsp;"name": "graphify-local",<br/>&nbsp;&nbsp;&nbsp;"transport": "streamable_http",<br/>&nbsp;&nbsp;&nbsp;
```

El token real no debe devolverse después de guardarse. La respuesta POST no debe incluir el secret en ningún campo.

## 18.2. Handshake

El gateway debe almacenar tras el handshake:

- Protocol version.

- Server name.
- Server version.
- Capabilities.
- Tools.
- Resources.
- Fecha de último healthcheck.

### 18.3. Scopes iniciales

```
graph.read<br/>graph.write<br/>memory.read<br/>memory.write<br/>calendar.read<br/>calendar.write<br/>
```

email.send, filesystem.write y acciones equivalentes requieren confirmación fuerte y no se habilitan por defecto.

### 18.4. Aislamiento

- Timeouts.
- Límite de tamaño de respuestas.
- Límite de llamadas por operación.
- Red permitida.
- No pasar todos los secretos del proceso a un MCP.
- Credenciales específicas por integración.
- Revocación.

## 19. LLM Router y contexto

### 19.1. Pipeline

```
Intent classifier<br/>&nbsp;&nbsp;&nbsp;<br/>Context retriever<br/>&nbsp;&nbsp;&nbsp;<br/>Prompt builder<br/>&
```

### 19.2. Contexto mínimo

El retriever debe enviar solamente:

- Memorias relevantes.
- Relaciones necesarias.
- Preferencias aplicables.
- Estado de tarea relacionado.
- Instrucciones de la skill.



## 21. Caching e índices

### 21.1. Redis

Usos de Redis en el backend:

- Rate limit.
- Locks distribuidos.
- Cola.
- Cache de consultas cortas.
- Estado temporal de confirmaciones.
- Pub/sub para actualización de UI.

No usar Redis como única fuente de verdad para memorias o recordatorios. Redis es cache y cola, no persistencia crítica.

### 21.2. Índices de base de datos

Índices recomendados:

```
memory_nodes(workspace_id, type)<br/>memory_nodes(workspace_id, created_at)<br/>memory_nodes(workspace_id, updated_at)
```

### 21.3. Índice vectorial

El vector debe asociarse al workspace y al nodo. Si el nodo se elimina, su vector debe invalidarse y purgarse para evitar búsquedas fantasma.

## 22. Notificaciones

### 22.1. Modelo

```
Notification<br/>- id<br/>- workspace_id<br/>- occurrence_id nullable<br/>- channel<br/>- payload_ref<br/>- status
```

### 22.2. Estados

```
pending<br/>sending<br/>delivered<br/>failed<br/>cancelled
```

### 22.3. Reglas

- No incluir secretos en previews de notificación.
- Permitir modo silencioso.
- Respetar preferencias de canal.
- No enviar una notificación duplicada por reintento.

- Mostrar error de configuración sin fingir entrega.

## 23. Exportación, importación y backups

### 23.1. Exportación

POST /exports

Debe ser asíncrona para volúmenes grandes. Contenido:

```
manifest.json<br/>user-settings.json<br/>memories.jsonl.enc<br/>edges.jsonl.enc<br/>reminders.jsonl.en
```

### 23.2. Importación

Debe:

- Validar schema.
- Verificar checksum.
- Mostrar resumen antes de importar.
- Permitir importar a workspace nuevo.
- Detectar duplicados.
- Mantener procedencia de importación.

### 23.3. Backup

- Backup lógico cifrado.
- Backup de base de datos según despliegue.
- Backup de objetos.
- Prueba periódica de restauración.
- No guardar la clave de descifrado junto al backup.

## 24. Healthchecks

GET /health/live

Responde si el proceso está vivo:

```
{<br/>&nbsp;&nbsp;&nbsp;"status": "ok",<br/>&nbsp;&nbsp;&nbsp;"service": "vnbot-api",<br/>&nbsp;&nbsp;&nbsp;"version": "
```

GET /health/ready

Comprueba dependencias críticas:

- Base de datos.
- Migraciones.

- Cola cuando el modo lo requiere.
- Storage si se necesitan archivos.

**GET /health/dependencies**

Devuelve estado detallado:

```
{<br/>&nbsp;&nbsp;&nbsp;"database": {"status": "ok", "latency_ms": 4},<br/>&nbsp;&nbsp;&nbsp;"redis": {"status": "ok", "latency_ms": 2}}
```

Los proveedores opcionales no deben hacer que una instalación básica aparezca como completamente caída. El healthcheck debe distinguir críticos de opcionales.

## 25. Errores y códigos

### 25.1. Categorías

AUTH\_\*      autenticación<br/>PERMISSION\_\*      autorización<br/>VALIDATION\_\*      payload inválido<br/>REMINDER\_\*      recordatorio

### 25.2. Ejemplos

AUTH\_SESSION\_EXPIRED<br/>PERMISSION\_TOOL\_DENIED<br/>VALIDATION\_INVALID\_DATETIME<br/>REMINDER\_AMBIGUOUS

### 25.3. Retryable

Cada error debe indicar si es reintentable. No reintentar automáticamente errores de autorización, validación o confirmación faltante. Los errores retryable son típicamente fallos de red, rate limits y caídas temporales de proveedores.

## 26. Seguridad del backend

### 26.1. Secretos

- Variables de entorno solo en desarrollo controlado.
- Docker secrets o vault en servidor.
- Keychain del sistema para desktop.
- No tokens en URL.
- No secretos en logs.
- Rotación documentada.

### 26.2. Archivos

- Validar tamaño.
- Validar MIME real.



- Renombrar con identificadores internos.
- Guardar fuera del web root.
- Analizar malware cuando el despliegue lo requiera.
- Impedir path traversal.

### 26.3. URLs externas

El fetch web y MCP deben protegerse contra SSRF:

- Bloquear redes privadas por defecto.
- Permitir allowlist configurada.
- Resolver y verificar IP final.
- Limitar redirects.
- Timeout.
- Tamaño máximo de respuesta.

### 26.4. Prompt injection

El contenido de emails, páginas, documentos y MCP debe tratarse como datos no confiables. Las instrucciones encontradas dentro de esos datos no deben modificar la política del agente.

## 27. Concurrencia y consistencia

### 27.1. Recordatorios

Usar lock por ocurrencia. La entrega debe ser exactamente una vez en condiciones normales y como máximo una vez en escenarios de recuperación no verificable, con deduplicación por provider cuando sea posible.

### 27.2. Memoria

Usar versionado optimista:

```
PATCH con expected_version
```

Si otro proceso modificó el nodo, devolver conflicto y no sobrescribir silenciosamente. El cliente debe refrescar y volver a intentar.

### 27.3. Integraciones

Cada integración debe guardar cursor o timestamp de sincronización y manejar eventos repetidos con idempotency keys.

## 28. Escalabilidad

### 28.1. Escalado vertical inicial

Para una instalación personal, aumentar CPU/RAM y mantener una sola instancia puede ser suficiente. La arquitectura permite empezar simple y crecer sin reescribir.

### 28.2. Escalado horizontal

La API debe ser stateless salvo por:

- Sesión server-side si se usa.
- Cache Redis.
- Base de datos.
- Storage externo.

Los workers pueden aumentar según la cola.

### 28.3. Cuellos de botella esperados

- Transcripción local.
- Embeddings.
- Proveedores LLM.
- Reindexación.
- Render del grafo en cliente.
- Entrega de notificaciones.
- Storage de audios.

### 28.4. Límites configurables

- Memorias por workspace.
- Tamaño de archivo.
- Audio diario.
- Jobs concurrentes.
- Tokens por usuario.
- Profundidad de grafo.
- Número de MCP.
- Herramientas por agente.

## 29. Configuración

### 29.1. Variables principales

```
VNBOT_ENV=production<br/>VNBOT_BASE_URL=https://vnbot.example.com<br/>DATABASE_URL=postgresql+asyncpg://
```

### 29.2. Reglas

- .env.example no contiene secretos reales.
- Configuración crítica debe validarse al iniciar.
- Proveedores opcionales pueden quedar desactivados.
- Un error de configuración debe indicar cómo corregirse.

## 30. Pruebas del backend

### 30.1. Unitarias

- Entidades.
- Recurrencia.
- Timezones.
- Policy engine.
- Idempotency keys.
- Parsers.
- Validación de tools.
- Cifrado.

### 30.2. Integración

- SQLite.
- PostgreSQL.
- Redis.
- Worker.
- Scheduler.
- Mock LLM.
- Mock MCP.
- Storage.

### 30.3. Contratos

- OpenAPI.
- JSON Schema de skills.
- Tool schemas MCP.
- Export/import.
- Eventos internos.

### 30.4. E2E

- Crear recordatorio y reiniciar worker.
- Reintentar delivery.
- Confirmar acción.
- Cancelar antes de ejecución.
- Borrar memoria y verificar índices.
- Desconectar MCP.
- Importar backup.

## 31. Observabilidad y privacidad

La observabilidad debe ayudar a operar el sistema sin convertir el contenido del usuario en telemetría. Esta dualidad es esencial: trazabilidad operativa sin invasión de privacidad.

### 31.1. Se puede registrar

- IDs internos.
- Tiempos.
- Estados.
- Códigos de error.
- Proveedor/modelo.
- Tokens agregados.
- Tamaños.
- Conteos.
- Hashes no reversibles cuando sean necesarios.

### 31.2. No registrar por defecto

- Plaintext de memorias.
- Audio.

- API keys.
- Tokens OAuth.
- Passwords.
- Contenido completo de emails.
- Respuestas completas de proveedores.

## 32. Requisitos para Docker

| Servicio  | Requisitos                                                                                                          |
|-----------|---------------------------------------------------------------------------------------------------------------------|
| API       | Imagen no root, Healthcheck HTTP, Migración explícita, Puerto interno, Sin persistencia local crítica.              |
| Worker    | Healthcheck de proceso, Conexión a Redis, Conexión a base de datos, Graceful shutdown, Visibilidad de jobs activos. |
| Scheduler | Lock distribuido, Una sola autoridad para crear ocurrencias, Graceful shutdown.                                     |

### 32.1. Volúmenes

```
postgres_data<br/>redis_data opcional<br/>minio_data<br/>vnbot_backups
```

## 33. Criterios de aceptación backend MVP

El backend cumple el MVP cuando satisface los 15 criterios:

01. Puede crear y consultar una memoria.
02. Puede crear un nodo y una relación.
03. Puede crear un recordatorio con zona horaria.
04. Puede generar ocurrencias recurrentes.
05. Puede entregar una notificación sin duplicarla.
06. Puede reintentar un job fallido.
07. Puede detener un job cancelable.
08. Puede procesar una instrucción sin LLM mediante heurística.
09. Puede registrar una operación y su auditoría.
10. Puede exportar/importar datos.
11. Puede ejecutarse con SQLite en local.
12. Puede ejecutarse con PostgreSQL y Redis en servidor.

- 13. Expone live/ready/dependencies.
- 14. No mezcla credenciales con datos de usuario.
- 15. No autoriza herramientas solo porque el LLM las solicite.

## 34. Orden de implementación backend

El backend se implementa en 7 fases secuenciales con criterios de salida:

| Fase                   | Módulos                                                                      | Criterio de salida                      |
|------------------------|------------------------------------------------------------------------------|-----------------------------------------|
| Fase 1 – Dominio       | Entidades, estados, reglas de recordatorio, memoria y grafo, tests unitarios | Tests unitarios pasan, cobertura 80%+   |
| Fase 2 – Persistencia  | SQLAlchemy, SQLite, migraciones, repositorios, exportación básica            | CRUD de memoria y recordatorio funciona |
| Fase 3 – API           | Auth, Memory, Graph, Reminders, Operations, Health                           | OpenAPI documenta todos los endpoints   |
| Fase 4 – Jobs          | Tabla/cola, Worker, Scheduler, Locks, Reintentos, Delivery                   | Recordatorio se entrega sin duplicados  |
| Fase 5 – IA            | Heurística, LLM Router, Structured output, Embeddings                        | Funciona con y sin LLM                  |
| Fase 6 – Integraciones | Notificaciones, MCP interno, MCP externo, Graphify, Calendario               | MCP con scopes y policy engine          |
| Fase 7 – Producción    | PostgreSQL, Redis, MinIO, Docker, Observabilidad, Backups, CI/CD             | Despliegue reproducible                 |

## 35. Decisiones abiertas

Estas decisiones deberán cerrarse antes de cada fase correspondiente:

- 01. Dramatiq o Celery como worker principal.
- 02. SQLite job queue propia o Redis obligatorio en modo local.
- 03. pgvector como único índice vectorial o adaptador adicional.
- 04. Transporte WebSocket o SSE para estados de jobs.
- 05. Duración de retención de operaciones y logs.
- 06. Política de soft delete por tipo de dato.
- 07. Soporte de múltiples regiones/zonas horarias en workspaces compartidos.
- 08. Formato definitivo de plugins externos.
- 09. Servicio separado de notificaciones en VNBOT 1.0.

10. WebAuthn en el MVP o en una versión posterior.

## 36. Conclusión

El backend de VNBOT debe ser una plataforma modular, no un conjunto de endpoints que llaman directamente a un LLM. La memoria, los recordatorios y las operaciones deben tener reglas deterministas y estados persistentes. La IA, MCP y las integraciones deben utilizar esas reglas, no sustituirlas.

La estructura recomendada es:

```
API stateless<br/>+ dominio explícito<br/>+ storage adapters<br/>+ jobs durables<br/>+ scheduler idemp
```

Esta arquitectura permite comenzar con SQLite y una instalación personal, y evolucionar hacia PostgreSQL, Redis, workers replicados y múltiples agentes sin reescribir el núcleo. También permite que VNBOT se distribuya en web, APK, desktop, Docker y CLI manteniendo el mismo modelo de datos y las mismas reglas de seguridad.

## 37. Endpoints de sincronización (fase 0.3+)

Estos endpoints se implementan cuando la sync strategy esté diseñada y probada.

```
GET /api/v1/sync/status
```

Devuelve el estado de sincronización del workspace.

```
POST /api/v1/sync/push
```

Envía cambios locales al servidor. El cuerpo incluye ops con version vectors.

```
POST /api/v1/sync/pull
```

Recibe cambios del servidor desde un cursor dado.

```
GET /api/v1/sync/conflicts
```

Lista conflictos pendientes de resolución.

```
POST /api/v1/sync/conflicts/{id}/resolve
```

Resuelve un conflicto específico con la opción elegida por el usuario.

```
POST /api/v1/sync/full-reset
```

Resetea la sincronización (peligroso, requiere confirmación doble).

## 38. Requisitos de testing del backend

### 38.1. Tests unitarios

Cada servicio de dominio debe tener tests unitarios:

- **MemoryService:** crear, actualizar, borrar, buscar, relacionar, olvidar.
- **ReminderService:** crear puntual, crear recurrente, disparar, completar, posponer.
- **GraphService:** agregar nodo, agregar edge, recorrido, invalidación.
- **AgentService:** crear, asignar skills, verificar permisos.
- **PolicyEngine:** evaluar permiso, denegar por defecto, auditar.
- **LLMRouter:** seleccionar provider, fallback, structured output parsing.

Cobertura mínima: 80% en dominio, 70% en API.

### 38.2. Tests de integración

- Crear recordatorio → verificar que se dispara en el momento correcto.
- Crear memoria → verificar que aparece en búsqueda.
- Conectar MCP tool → verificar que se ejecuta con permisos correctos.
- Exportar e importar → verificar integridad de datos.

### 38.3. Contract testing para MCP

Cada tool MCP debe tener un contract test que verifique:

- Input schema es válido.
- Output schema es válido.
- La tool respeta sus scopes declarados.
- Un fallo de la tool no afecta al núcleo.

### 38.4. Benchmarks automatizados

- Operaciones CRUD sobre memorias (1,000 ops).
- Búsqueda textual con 5,000 memorias.
- Recordatorios: crear 1,000 y verificar que ninguno se duplica.
- Grafo: subgrafo profundidad 3 con 50 nodos.
- Scheduler: 100 jobs concurrentes sin duplicados.



## 39. Instrumentación de observabilidad

Todos los servicios del backend deben estar instrumentados con OpenTelemetry desde la primera versión.

### 39.1. Spans obligatorios

- **vnbot.api.<method>.<endpoint>**: cada request HTTP.
- **vnbot.domain.<service>.<operation>**: cada operación de dominio.
- **vnbot.llm.<provider>.call**: cada llamada a LLM.
- **vnbot.mcp.<tool>.call**: cada invocación de tool MCP.
- **vnbot.worker.<job\_type>.execute**: cada ejecución de job.
- **vnbot.sync.push / vnbot.sync.pull**: cada operación de sincronización.

### 39.2. Métricas obligatorias

vnbot\_api\_requests\_total{method, endpoint, status}<br/>vnbot\_api\_duration\_seconds{method, endpoint} (h

### 39.3. Regla de contenido

Ningún span, métrica ni log debe contener el contenido de una memoria, mensaje o dato personal. Solo IDs, estados, duraciones y tipos. Esta regla se valida con scanners automáticos de PII en CI.

## 40. Restricciones de rendimiento del grafo

### 40.1. Límites por consulta

| Parámetro                            | Valor               |
|--------------------------------------|---------------------|
| Profundidad máxima por defecto       | 3                   |
| Top-K máximo configurable            | 50 (por defecto 20) |
| Nodos máximos devueltos por consulta | 100                 |
| Timeout por consulta                 | 5 segundos          |

### 40.2. No cargar el grafo completo

El cliente nunca recibe el grafo completo. Todas las consultas son server-side con paginación y filtros. La visualización recibe solo los nodos y edges del resultado.

### 40.3. Degradación graciosa

Si una consulta excede los límites, el sistema sigue 4 pasos:

01. Reduce la profundidad.
02. Aplica top-K más agresivo.
03. Devuelve un mensaje al usuario: "La consulta es muy amplia. Refina los filtros o selecciona un nodo de inicio."
04. Nunca agota la memoria del servidor por una consulta de grafo.